Product Requirement Document (PRD)

1. Overview & Objectives

The Automated Financial Expiry Countdown (AFEC) app is a privacy-focused personal utility designed to automatically log and monitor expiration timelines (Fixed Deposits, premium accounts, and card renewals) by scanning inbound financial SMS. It strips out complex budgeting matrices and focuses strictly on visual time horizons and multi-stage proactive alerts to ensure users take timely action.

Target Audience

- Individuals managing multiple Fixed Deposits (FDs) across different banks.
- Users who frequently miss account renewal timelines due to manual logging friction.
- Privacy-conscious users who prefer local-first data processing.

2. Key Features & Scope

In-Scope (Phase 1)

- **Automatic SMS Parsing**: Background monitoring of text messages to detect financial dates (FD maturity, debit card expiry).

- **Visual Countdown Dashboard**: Big, scannable countdown cards showing remaining days at a glance.
- **Notification Stacking (Multi-Alerts)**: Fully customizable pre-alerts (e.g., 7 days before, 3 days before, and day-of).
- **Manual Entry Fallback**: Simple form to manually input or override tracking dates.
- **Local-First Architecture**: Local database storage (Room DB) with zero cloud synchronization to protect financial data privacy.

Out-of-Scope (Future Phases)

- Automatic account balance tracking or expense categorization.
- Direct integration with bank APIs (NetBanking/Open Banking).
- Cloud backup/Sync across multiple devices.

3. User Experience & Core Screens

Screen 1: The Countdown Dashboard (Primary View)

- **UI Layout**: A vertical scrolling list of color-coded cards.
- **Card Elements**:
  - Bold title: e.g., "HDFC FD Withdrawal" or "SBI Account Renewal"
  - Center metric: Big visual number showing "XX Days Left"
  - Sub-text: Exact maturity date and principal amount (if parsed).
- **Interaction**: Tapping a card reveals action buttons: [Mark Completed / Renew] or [Edit Details].

Screen 2: SMS Verification Queue (The Inbox Log)

- **Purpose**: When the background script extracts a date, the item is sent here first to prevent cluttering the main screen with false positives.

- **UI Element**: A simple list showing the raw SMS text next to the parsed result.
- **Action**: User taps [Approve & Add to Dashboard] or [Dismiss].

4. Technical Requirements & Architecture

Permissions Framework

- **RECEIVE_SMS & READ_SMS**: Required immediately upon first launch to process automated date capturing.
- **POST_NOTIFICATIONS**: Required on Android 13+ to ensure time-critical alarms fire reliably.

Data & Architecture

- **Database**: Android Room SQLite for persistent storage on the device.
- **Background Processing**: Android WorkManager combined with standard hardware alarms (AlarmManager) to trigger alerts precisely at 9:00 AM on user-designated countdown milestones, even if the app is closed.

5. Functional Requirements Matrix

Feature ID	Feature Name	Description	Priority
FR-01	SMS Ingestion	Automatically monitor incoming texts for keywords: matures, maturity, expiry, value date, lock-in.	P0 (Critical)

FR-02	Regex Date Engine	Parse variable date strings safely (e.g., DD/MM/YYYY, DD-MMM-YYYY, DDth Month YY).	P0 (Critical)
FR-03	Visual Countdown	Sort dashboard automatically by closest maturity date first. Display dynamic day counters.	P0 (Critical)
FR-04	Notification Stacking	Provide a configuration menu for custom intervals (e.g., 14 days out, 7 days out, 1 day out).	P1 (High)
FR-05	One-Click Reset	An entry rollover button that pushes a recurring item to its next yearly iteration instantly.	P2 (Medium)

6. Constraints & Implementation Risks

- **Google Play Policy Restrictions**: Google highly restricts public distribution of apps requiring broad READ_SMS permissions. This product is designed to be directly sideloaded as a private developer build via Android Studio or an APK compiler to bypass store policy blockades.
- **Bank SMS Variations**: Bank statement text formats change frequently. The internal keyword parser should support manual text updating or fallback to manual approval whenever string matching scoring is low.

Would you like me to generate the ready-to-compile Android code framework (Kotlin/Jetpack Compose) or build out the specific Regex processing code for Indian banking SMS templates